USER STORY · 三来源 · 全生命周期

超级智能体的一天

从 Kanban 看板、Gitea 仓库到 ntfy 主题——
一条消息如何变成一次被授权、可执行、能回写的交付。

Kanban · 任务 | Gitea · 代码 / Issue | ntfy · IM 提醒 | Human-in-the-loop

角色
小吴 · 开发负责人

流水线
Source → Trigger → Eval → Ticket → Executor → Channel

文档
STORY.md → REPORT.md → TDD/007

SUPERAGENT

ROADMAP

今天要讲清的四件事

01 · SCENE

Kanban 看板任务

拖进 ready 列之后，从 Trigger 到工单、到评论回写、再到看板推进的完整闭环。

02 · SCENE

Gitea：事件仓 + 宪章仓

事件仓的 Issue/PR 只消费增量、把提及变成请求；**宪章仓的 README.md 是宪章的家**——两个仓库分开。

03 · SCENE

ntfy：带 tag 的消息才入库

只有带**入库 tag**（如 `dsh`）的消息才变成看板卡；不带 tag 的只是聊天，宿主的回复也不会被回收。

04 · MODEL

全生命周期与人类方向盘

十个阶段一张表，加上人类可随时按下的暂停、停止、重试与测试按钮。

一个人，三条消息流

超级智能体没有自己的密码库，它借用团队给的机器人身份，在三个来源上“听”和“说”。

SOURCE · KANEO

Kanban 看板

pl.feg.cn · FDEP 项目

监听列可配（本部署 **to-do**）；任务 = Thread，回复 = 任务评论。

`poll 60s` `API key`

SOURCE · GITEA

Gitea 仓库

git.feg.cn · 事件仓 + 宪章仓

Issue / PR / 评论增量；回复 = Issue 评论。宪章仓的 README.md = 项目宪章。

`email + password` `增量` `宪章提交`

SOURCE · NTFY

ntfy 主题

msn.feg.cn · topic

只有带入库 tag 的消息入库成卡；回复发布回同一 topic。

`tag 过滤` `pub_ 公开` `私有可账号密码`

三类来源共享同一条流水线：**Source → Trigger → Thread → Eval → Mandate → Ticket → Assignment → Executor → Channel → Outbox**

ONE PIPELINE

一张图看懂：消息如何变成交付

来源 Kanban / Gitea / ntfy → **触发** 归一化事件 → **Gate** 名单 / 提及 / 准入 → **研判** 主 Session 决定 → **授权** Mandate → **工单** Ticket

派工 Assignment → **执行** 隔离 worker → **回复** Channel → **回执** Outbox 幂等 → **结束** ended / 退订

它只做被授权的事

Gate 先把消息分成 explicit / inferred / ignore；没有明确授权的，不会变成工单。

它在自己的房间里干活

Executor 是独立 Session：独立目录、受限命令、受控预算，主 Session 只看结果。

它把结果送回原处

评论回 Kanban / Gitea，消息回 ntfy；幂等标记保证重试不会刷屏。

SCENE 01 · 09:10 — 09:16

Kanban：拖进 ready 之后

09:10

一张卡被拖进 ready

“修复登录页在 1366×768 下的横向滚动”——对人是拖一下，对流水线是起点。

09:11

Source → Trigger → Gate

轮询发现新卡，归一化为 Trigger（线程 = 任务）；小吴在允许名单且进入监听列 → **explicit**。

`kaneo:<taskId>` `actor = 小吴` `addressed = 明确请求`

09:12

研判 → Mandate → Ticket

主 Session 判断“范围清楚、可执行”，记录授权依据，创建工单；看板卡片按配置推进到 in-progress。

09:16

Assignment → Executor → Channel

隔离 worker 只改样式并跑测试；结果作为**任务评论**写回，带幂等标记；工单 ended · completed。

`POST /comment/{taskId}` `<!-- dsh-superagent:… -->`

小吴看到的不是日志，而是卡片下的一条评论与一个状态变化。

SCENE 02 · 10:05

Gitea：Issue 里 @一句 就够

发生了什么

- 同事在 **shawoo/PDP** 提 Issue：“@shawoo 麻烦看一下，和 #12 可能同因。”
- Source 只读**增量**：issues / comments 各带游标，前移不重复。
- ✓ 提到 agent 登录名 → **addressed: mention**；作者在 allowed 名单 → explicit。
- ✓ 工单 → worker 排查 → 结果以 **Issue 评论** 写回。
- ✓ 宪章在**另一个仓库**：**shawoo/rsi_superagent** 的 README.md 就是项目宪章——发布 = 提交该文件，commit 历史即版本历史。

边界与细节

- 没被 @ / 不在名单 → 默认 ignore（可配置 inferred）。
- 同一 Thread 的追问 → 上下文续传给执行中的 Assignment。
- 账号用**邮箱 + 密码（HTTP Basic）**；密码框掩码、接口不回显。
- ✓ 改完账号先点「测试连接」，通过才允许保存。
- **事件仓与宪章仓是两个字段、两个仓库**，必须分开；宪章仓留空则退回 Kanban 项目描述镜像。

增量轮询 since 游标 → **提及识别** @login → **工单** Ticket → **评论回写** 幂等

发布宪章 = 向宪章仓库 README.md 提交 `charter vN: <发布者>`；岗位开工先 `orchestrator_read charter` 读当期宪章。

SCENE 03 · 14:30

ntfy：公开 topic 低信任，私有可账号密码

消息到达

title：#wu.zhiping
body：@dsh-superagent 线上告警：支付回调 500 突增，先看最近一次发布是否有异常。

poll 30s | #name = 身份 | @agent = addressed | tag 入库

信任边界

pub_ 公开 topic：**无认证**，title 里的身份可被冒用；私有 topic 可用**账号+密码（Basic）或 token** 保护，只有持凭据者能发布。

- ✓ allowed 名单里的发送者 + @ 智能体 → **explicit**
- • 否则 → inferred（observe 只记录）/ ignore
- ✗ 绝不因为“消息说自己是小吴”就当成授权

回复也回到 topic

标题 #dsh-superagent，带幂等 marker；它的 tag 不会被入库过滤收成卡。

入库过滤 tag

只有带**入库 tag**（本部署 dsh）的消息才入库成卡；不带 tag 的留在 topic 当聊天，看板不再被闲聊塞满。

生产建议

优先私有 topic + **账号密码（Basic）或 token**；公开 pub_ 仅测试/低风险提醒；公开 topic 的低信任来源不拥有最高授权。

HUMAN IN THE LOOP

人始终握着方向盘

人的动作	系统反应
改来源账号	先「测试连接」，通过才保存；保存后来源重启，health 变为 ready。
改 Policy 名单 / 准入	立即生效：之后的消息按新规则被判 explicit / inferred / ignore。
暂停 Pause	不再产生新工单；排队效果停住，恢复后继续。
停止某张工单	Ticket 结束，worker 收尾，不再回复。
重试失败工单	从失败点重新派工，来源线程不重复消费。
在 Thread 追问	新消息作为上下文续传给执行中的 Assignment；结束后自动退订。
查看决策 / 工单 / 执行面板	每条 Trigger 是接单、过滤还是仅候选，都有原因可查。

全生命周期：从部署到研判

阶段	输入	系统动作	产出	失败 / 恢复
部署检查	账号、stateDir、executor	在线探测 Kanban / Gitea / ntfy，CLI 校验 GitHub / Feishu	Deployment	面板显示修复入口；health paused / ready / error
轮询接收	来源增量	读游标、首次记 baseline；每个流独立容错	Trigger	失败计数达阈值报 error；游标不丢
Gate 策略	Trigger + Policy	名单 + addressed + 准入矩阵；bot 至多 inferred	explicit / inferred / ignore	决策记录可查；ignore 不打扰
Eval 研判	明确/候选 + 上下文	主 Session 判断范围、可行性、风险	Decision	原因、耗时、规则命中可追溯
Mandate 授权	接单决定	记录“谁、哪条消息、什么范围”	Mandate	审计可回答“凭什么做”

全生命周期：从工单到恢复

阶段	输入	系统动作	产出	失败 / 恢复
Ticket 工单	授权	绑定 Thread、选定 Channel 地址、去重合并	Ticket open	后续消息挂到已有工单
Assignment 派工	Ticket + 执行配置	隔离 worker Session、目录、环境变量、工具白名单	Assignment	可重试 / 停止；结果回主 Session
Executor 执行	派工	读上下文、干活、回报 ready / blocked / failed / dropped	结果与产物	与主 Session 隔离；预算 / 并发受控
Channel 回复	结果	转评论 / 消息，加幂等标记，落 Outbox	来源回执	失败退避重试；重试不重复发
结束 & 反馈	回执、工单终态	ended、退订 Thread、按配置推进看板列	完成状态	面板可审计
恢复	重启 / 抖动 / 来源删除	游标续读、Outbox 幂等重放、partial 不毒化	无需人工重放	baseline + 游标 + 回执：不丢不重

RESILIENCE

韧性：不丢、不重、不卡死

不丢

游标 + baseline

首次轮询只记基线；之后按游标只读增量。重启后继续，不重放历史。

不重

Outbox + 幂等标记

回复先入持久队列；发送前查最近消息是否已有 marker，重试不会在来源刷屏。

不卡死

每个流独立容错

issues / comments / 订阅线程分别 try-catch；一个流坏了不毒化整条来源，partial 也照常推进。

可观测

health 与决策记录

每个来源有 health、统计、过滤原因；每条决策都能回答“为什么接 / 为什么不接”。

SCOPE

能做 / 不做：同一故事的约束

IT DOES

- ✓ 监听三类来源增量，按策略把明确授权变成工单
- ✓ 在隔离 Session 执行，把结果写回来源
- ✓ 幂等、可审计、可暂停、可重试
- ✓ 账号/仓库变更前先做在线测试

IT DOES NOT

- ✗ 把公开 pub_ topic 上的“我是谁”当成授权
- ✗ 在未获授权时修改用户仓库
- ✗ 绕过 blocked commands / 预算 / 并发上限
- ✗ 因重试或重启在来源重复刷评论
- ✗ 无工单、无回执就宣称“已完成”

FIRST DEPLOYMENT

小吴的第一次部署 · 5 步

01 装 bundle
设置 stateDir

→

02 配四来源
+ MinIO 卡

→

03 测试连接 + 发宪章
宪章仓 README

→

04 定 Policy
allowed + 信任策略

→

05 验收
health 全 ready

Kanban

服务器 + API key + 项目 id；保存前自动探测，也可点「测试连接」。

Gitea

服务器 + 邮箱 + 密码（或 token）+ **事件仓 owner/name** + **宪章仓 + 宪章文件**；**测试通过再保存**。

ntfy

服务器 + topic + 认证：公开 pub_ 选 none；私有 topic 用账号+密码（basic）或 token。

验收：/orchestrator/api/state 中 runtime / kaneo / gitea / ntfy / s3 全部 ready；**切到 Team 视角先过权限自检**；宪章未发布时项目不接受任务（没有宪章不可开工）；再让一条真实消息走完全流程。

NEW • 2026-11-06

宪章有自己的仓库，任务、消息、资产各有载体

一个载体一件事；凭据默认取本机存储记录（Masked Key），卡片可逐字段覆盖。

环节	载体	规矩
宪章	Gitea **宪章仓库**的 README.md（与事件仓库分开）	发布 = 提交该文件；commit 历史 = 版本历史；编队页徽标显示载体
任务	Kanban 看板（监听列可配，如 to-do）	卡片是唯一派工来源；没有宪章的项目不接受任务
实时消息	ntfy 项目 topic pub_<项目 id>	只有带入库 tag 的消息才入库成卡；宿主回复 tag dsh-superagent 不会被回收
数据资产	「MinIO 对象存储」来源卡	非代码交付物 deliverables/<项目 id>/<ticket id>/<file>，卡片只留进度与结果

Team 视角要先验权限

切到 Team（PM）视角前查 canManageFlow；缺权限先说明再让你决定。岗位视角的「我的岗位设置」只改自己的岗位。

岗位自己读宪章

orchestrator_read charter 返回版本、载体与正文——不需要 shell/HTTP，也不给岗位 Gitea 凭据。

没有宪章不可开工

评估接受与显式派工都会被拦成带 noCharter 理由的忽略；团队页红色提示，没有工单被创建。

一次值流闭环：4 分 40 秒，然后改流程、发新宪章

读宪章 v11 Gitea README → **派卡** RSI · to-do → **ntfy 入库** 9 秒 → **工单** 19 秒 → **MinIO 交付** 2 分 41 秒 → **卡片 done** 4 分 40 秒

复盘 · 发现的缺口

- ✗ 岗位读不到 Gitea 宪章（无 shell/HTTP、无 gitea 资源）→ 新增 `orchestrator_read charter`，第二张卡实测读回 v11。
- ✗ ntfy 换 topic 时游标可能跳过刚发的消息 → 换 topic 回退 600 秒（入库标记去重）。
- ✗ 交付物 key 两种写法并存 → 只保留 `deliverables/` 一种。

改进 · 写进哪里

- ✓ 流程改进 → **SOP**（§3.6 值流闭环演练 + §4.1 开工先读宪章）。
- ✓ 规则改进 → **宪章 v12**（提交到 `shawoo/rsi_superagent/README.md`，narrative 读回 `carrier=gitea`）。
- ✓ 能力改进 → `orchestrator_read charter` 与新来源卡（MinIO / ntfy tag / 宪章仓独立）。

TAKEAWAY

授权 执行 回写

超级智能体不是聊天窗口，而是一条可授权的流水线。

来源只是入口；真正决定它价值的是边界：**谁能发起、什么范围、结果如何回到原处。**

进一步阅读：STORY.md　设计：REPORT.md　实现与验证：TDD/007　领域模型：docs/001-domain.md